

EVMC

A functional systems language for Ethereum

An interpreter on Ethereum!

[illegible]

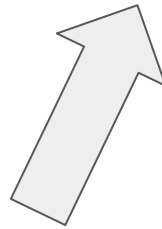
Ethereum ledger state: Address => Account

Alice => value: x, nonce: a

Bob => value: y, nonce: b

Charles => value: z, nonce: c, storage: {0=>...,1=>...}, bytecode: 0x...

???



Ethereum transactions, simplified

- **Call(address,gas,ether,calldata)** –pay EOA or call contract
- **Create(gas,ether,bytecode)** –create a new contract

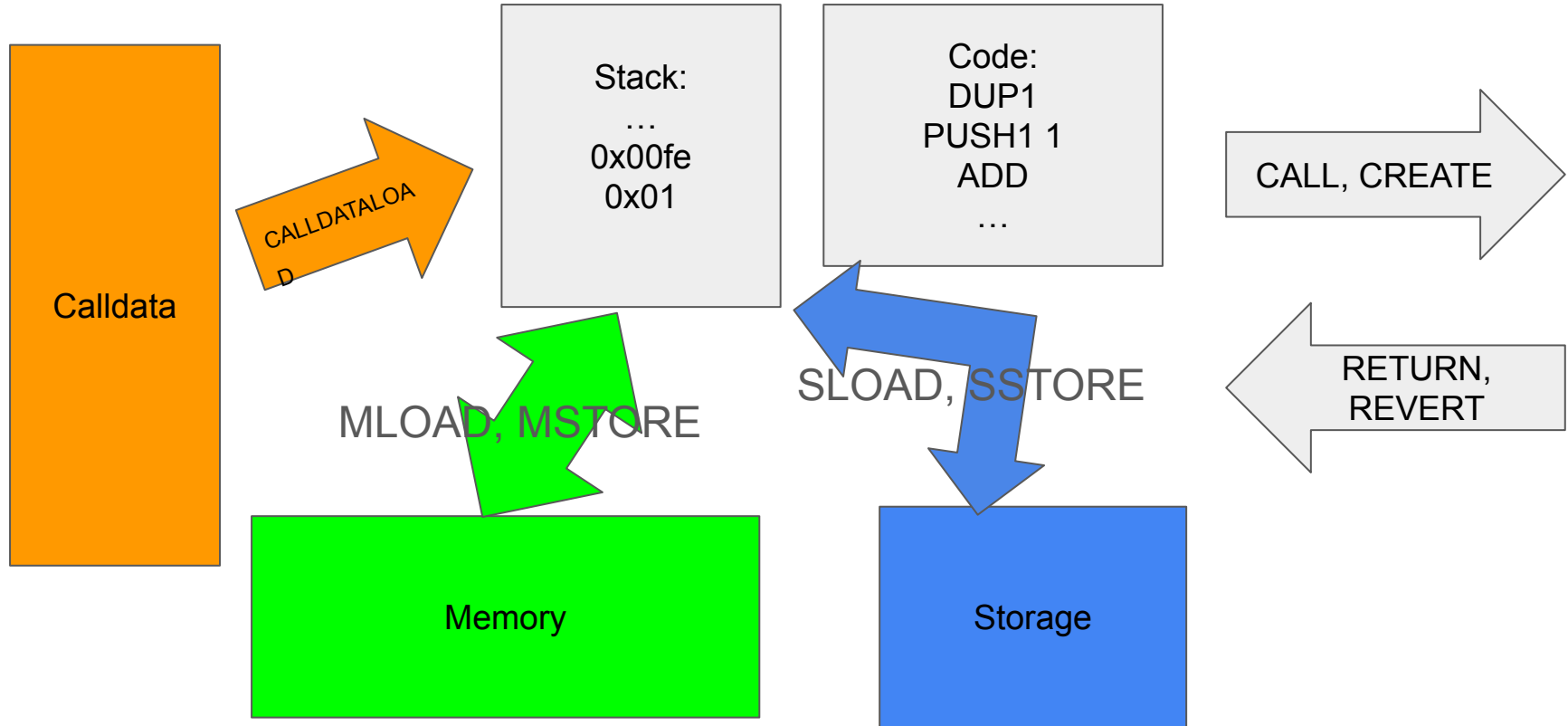
Ethereum smart contracts

On CALL: run its code.

Code can:

- Read and write private storage
- CALL other contracts
- CREATE other contracts
- RETURN a bytestring or REVERT

The EVM



EVM cost model

30M gas/block, 1 block/12s. 4M gas ~ \$1

Upload a byte: 16 gas

ADD, DUP*, SWAP*, POP, MSTORE, MLOAD: 3 gas

SSTORE: 5,000 gas

CALL: 2,600 gas

CREATE: >32,000 gas

Transaction: 21,000 gas

Ethereum is cool!

- First and largest **smart contract** blockchain
- Smart contracts = neutral programs that can hold and move money
- Used for stablecoins, DEXes, options, futures...
- \$295B market cap, \$127B TVL, largest dev ecosystem, institutional interest...

Smart contracts are tricky

- Bugs => \$millions lost
- Ethereum is less powerful than a Nokia from 1998*
- \$1 per 1.4M ADDs

Correctness and performance are **both** vital!

```
contract Coin {
```

```
...
```

```
mapping(address => uint) public balances;
```

```
function send(address receiver, uint amount) public {
```

```
    if (amount > balances[msg.sender])
```

```
        revert();
```

```
    balances[msg.sender] -= amount;
```

```
    balances[receiver] += amount;
```

```
}
```

```
}
```

The contract-oriented paradigm

Solidity contracts are objects with methods and state variables.

Contract creation => CREATE (>32000 gas)

Method call => CALL (2600 gas)

State variable write => SSTORE (5000 gas)

Each method call transaction: 21000 gas

Security through isolation?

All contracts are CALLable: no hidden objects!

Contracts can't trust their callers **or callees!**

All public methods available all the time

Protocols => public methods



PL

~~NOKIA~~
Connecting People

Interpreters let users trust and share

Batchers: share the 21,000 transaction cost

Batched method handling: share the CALL cost.

FW: avoid CALL, CREATE and SSTORE.

Can existing languages express them?

Solidity, Vyper:

- Inefficient ABI
- Limited type system
- No switch/case, inductive types, algebraic datatypes...
- Locks you out of the machine

Yul, Huff:

- No limits*...
- But no type system: everything is a word

Enter EVMC!

```
sum10 : Ptr r (Int s l) -> Int s l;
```

```
sum10 p := {
```

```
  var accum = 0,
```

```
    bound = indexPtr(p,10);
```

```
  while(coerce p < coerce bound)
```

```
    accum += *(p++);
```

```
  return accum
```

```
};
```

```
main():= returnValue (sum10 (null() :: Ptr Calldata Byte))
```

It's real!

```
> t10 = await u.deploy("TestSum10.evm")  
'0x5FbDB2315678afecb367f032d93F642f64180aa3'  
> await u.call({to: t10, data: "0x010101"})  
'0x03'  
> |
```

allocValue : a -> Ptr r a;

allocValue a := {

var p = alloc 1;

*p = a;

return p

};

```
alloc : Short -> Ptr r a;
```

```
instance alloc : Short -> MPtr a where
```

```
  n := {
```

```
    var sz = sizeof (P :: P a),
```

```
        m = sz * n; //The product gets masked here...
```

```
    return coerce (allocRawMem (coerce m)) :: MPtr a
```

```
};
```

```
memAllocPtr : Word;
```

```
memory memAllocPtr;
```

```
//Fateful decision: no overflow checking is done.
```

```
allocRawMem : Word -> MPtr Byte;
```

```
allocRawMem n := {
```

```
  var pw = memAllocPtr;
```

```
  memAllocPtr = n + pw;
```

```
  return unsafeCoerce pw
```

```
};
```

A kind system!

Type : Kind;

Nat : Kind;

Region : Kind;

Memory : Region;

Storage : Region;

Calldata : Region;

Code : Region;

Signedness : Kind;

Signed : Signedness;

Unsigned : Signedness;

Algebraic datatypes!

```
data Bool = {False;True};
```

```
data Maybe a = {Nothing;Just{fromJust: a}};
```

```
//The only boxed type provided by Prelude (for now):
```

```
data List r a = {Nil;Cons{hd:a,tl:List r a}} region r
```

Solidity compatibility

//All Solidity methods belong to an interface:

Interface : Kind;

Contract : Interface -> Type;

data Contract iface = {Contract {unContract: Addr}};

Method : Interface -> Type -> Type -> Type;

data Method iface arg ret = {Method (UInt 4)};

Solidity compatibility

`abi_call_method : (Contract iface, Method iface a b, a) -> b;`

`abi_handle_method : (Method iface a b, a -> b) -> ();`

Closures

```
data Hidden = {};
```

```
data Closure a b =
```

```
{Closure {
```

```
  c_fptr : (MPtr Hidden, a) -> b,
```

```
  c_closed : MPtr Hidden
```

```
}
```

```
};
```

Closures

`mkClosure : ((MPtr c, a) -> b, MPtr c) -> Closure a b;`

`mkClosure (f,c) := return Closure {c_fptr: coerce f, c_closed: coerce c};`

`callClosure : (Closure a b, a) -> b;`

`callClosure (c,a) := return c.c_fptr(c.c_closed,a);`

Objects

```
Object_ : (Type -> Type) -> Type -> Type;
```

```
data Object_ mtbl c = {Object_ {o_mtbl: MPtr (mtbl c), o_closed: c}};
```

```
Object : (Type -> Type) -> Type;
```

```
data Object mtbl = {Object {unObject: MPtr (Object_ mtbl Hidden)}};
```

```
OMethod : (Type -> Type) -> Type -> Type -> Type;
```

```
data OMethod mt a b = {OMethod Short};
```

Objects

```
mkObject : (MPtr (mt c), c) -> Object mt;
```

```
mkObject(mtp,c) :=
```

```
  return coerce (allocValue Object_ {o_mtbl: mtp, o_closed: c});
```

```
obj_call_method : (Object mt, OMethod mt a b, a) -> b;
```

```
obj_call_method (o,m,a) := {
```

```
  var moff = add(coerce (o.unObject->o_mtbl), coerce m);
```

```
  return (*(unsafeCoerce moff))(&(o.unObject->o_closed),a)
```

```
};
```

Inheritance!

```
OInherits : (Type -> Type) -> (Type -> Type) -> Type;
```

```
data OInherits parent child = {};
```

```
o_super : Object des -> Object anc;
```

```
o_super o := return o_superclass(oinherits(),o);
```

```
o_sub : OMethod anc a b -> OMethod des a b;
```

```
o_sub m := return o_subclass(oinherits(),m);
```

Inheritance!

```
oinherits : () -> OInherits mt1 mt2;
```

```
instance oinherits : () -> OInherits mt mt where
```

```
  _ := {};
```

```
ovia : (OInherits a b, OInherits b c) -> OInherits a c;
```

```
ovia _ := {};
```

Inheritance!

```
instance oinherits : () -> OInherits (GetTbl a) (RefTbl a) where  
  _ := {};
```

```
instance oinherits : () -> OInherits mt (FreezableTbl a) where  
  _ := return ovia(oinherits(),  
    null() :: OInherits (RefTbl a) (FreezableTbl a));
```

Thunks!

```
data Thunk a = {Thunk (MPtr (Thunk_ a))};
```

```
data Thunk_ a = {
```

```
    Unevaluated (OutChan a);
```

```
    Evaluating;
```

```
    Evaluated (MPtr a)
```

```
};
```

```
evalThunk : Thunk a -> a;
evalThunk (Thunk thp) := {
  var oc, pa;
  case *thp of {
    Unevaluated oc => {
      *thp = Evaluating;
      var a = readChan oc;
      *thp = Evaluated (allocValue a);
      return a
    };
    Evaluating => error "Encountered a cyclical dependency in thunk!";
    Evaluated pa => return *pa
  }
};
```

Coroutines!

```
data Coroutine = {Co {unCo: MPtr Coroutine_}};
```

```
data Coroutine_ = {Co_ {
```

```
  co_status: CoStatus, //indicates whether coroutine is blocked, running or done
```

```
  co_max_words: Short, //maximum number of words allocated for stack
```

```
  co_bottom: MPtr Word, //points to the bottom of the stack
```

```
  co_sentinel: ResumeSentinel //The in-memory sentinel, not part of the stack.
```

```
}
```

```
};
```

Coroutines!

```
newCo : (Short, s -> (), s) -> Coroutine;
```

```
yieldCo() := {
```

```
    var cop = co_self.unCo;
```

```
    cop->co_status = CoBlocked;
```

```
    return (coerce (&yieldAsm))(inc cop);
```

```
};
```

```
resumeCo : Coroutine -> ();
```

Assembly as a library!

```
code yieldAsm = Struct (  
  JUMPDEST,  
  DUP2, DUP2, MSTORE,  
  Push1 32, ADD,  
  SWAP1, Push2 (coerce (&coReturn)), XOR,  
  Push2 (coerce (&yieldAsm)),  
  JUMPI,  
  SWAP1, JUMP  
)
```

LLM interpreter: state

```
data Regs = {Regs {  
    pc: Word, //The program counter; interprets from calldata  
    accum: Word, //The accumulator register, used as the target for most ops  
    //Supplementary registers  
    r1: Word,  
    r2: Word,  
    r3: Word  
}  
};
```

LLM interpreter: instruction set

```
data Instr r = {  
  //Load a 16b constant into r_accum  
  Load_k Short;  
  //Calldataload off a fixed offset  
  CDL_k Short;  
  //Mload off a fixed offset  
  ML_k Short;  
  ...  
} region r;
```

LLM interpreter: dispatch loop

```
dispatch rs := {
```

```
  while (True) {
```

```
    var sh;
```

```
    case unsafeCoerce (rs.pc) :: Instr Calldata of {
```

```
      Load_k sh => {rs.accum = coerce sh; rs.pc+=3};
```

```
      CDL_k sh => {rs.accum = calldataload(coerce sh); rs.pc+=3};
```

```
    ...
```

Master plan: HLM

An interpreter for a functional language:

Arrays, balances, closures, abstract datatypes...

Virtual smart contracts: write in a HLL, communicate in memory!

Merkalized heap: all writes => a single SSTORE!

EVMC FW

Language improvements:

UX (pretty-printed compiler errors)

Spec, test suite;

More opts: aggressive inlining, better memory model, symbolic simplification, tail-call optimization...

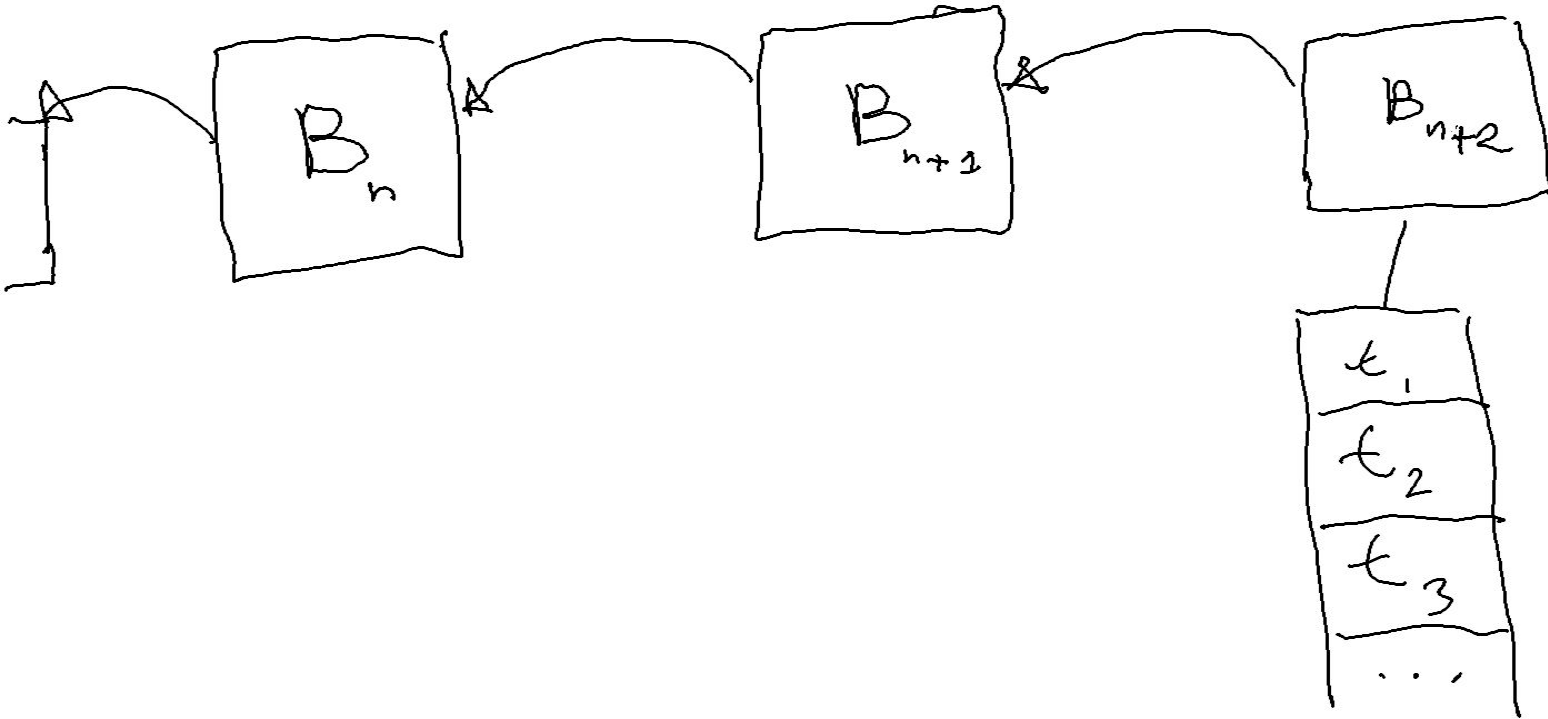
Annotations: `@callable f : T`, `@executable g`

...and much more.

IFL paper!

Thank you for listening!

Blockchains illustrated



Blockchain state, simplified

State(0) = <initial>

State(N) = runBlock(Block(N), State(N-1))

runBlock(block, state) =

for txn in block:

state = runTxn(txn, state)

return state